MONITOR Z · L15.3

Clase 3 — Los registros, la base y el desplazamiento

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L15 · Clase 3 — Los registros, la base y el desplazamiento

En la Clase 2 viste que el programa no sabe en qué dirección real lo va a cargar z/OS. Esta clase muestra cómo llega igual a sus datos y a sus etiquetas: con un **registro base** y un **desplazamiento**. De acá sale el tope de 12 KB que obliga a partir ZTKMON en módulos.

> 📚 Fuentes: el listado del ensamblado de **JOB34775** (`monitor_z/salidas/JOB34775/…/ASM/SYSPRINT.txt`): el prólogo (sentencias 10126–10137), las instrucciones 10189, 10199 y 10202, y el *Using Map* (página 227). *HLASM V1R6 Programmer's Guide* (SC26-4941): cap. 2 (USING map) y apéndice E (`ASMA034E`). *z/Architecture Principles of Operation* (SA22-7832), cap. 5 («Operand-address generation»).

1. Los registros: las 16 manos de la CPU

- Son **16** (R0 a R15) y cada uno guarda **4 bytes**. Con `AMODE 31` (Clase 2), una dirección entra en un registro.
- Las instrucciones trabajan con lo que tienen en las manos: cargan un dato de memoria a un registro (`L`), lo usan y lo

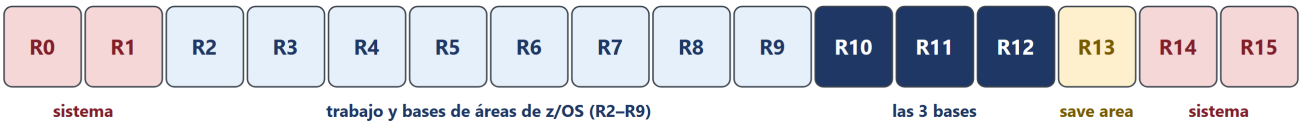
guardan (`ST`).

- **No todos están libres.** Así se reparten en ZTKMON:

Registros	Quién los usa	Por qué
R0, R1, R14, R15	el sistema	los pisan las macros y las llamadas (`STORAGE` devuelve la dirección en R1 y el RC en R15)
R13	la *save area*	lo pide la convención de enlace (L01; se ve de nuevo en la Clase 5)
R12, R11, R10	**las tres bases del programa**	`USING ZTKMON,R12,R11,R10` (sentencia 10132)
R2–R9	el trabajo, y las bases de las áreas de z/OS	por ejemplo `USING SSOB,R2` y `USING CVT,R1` (Clase 6)

Los 16 registros y quién usa cada uno

L15 · Clase 3 · Registros, base y desplazamiento



Cada registro guarda 4 bytes: con AMODE 31 (Clase 2), una dirección entra en un registro.

Registros	Quién los usa	Testigo en ZTKMON
R0 R1 R14 R15	el sistema: los pisan las macros y las llamadas	`STORAGE OBTAIN` devuelve la dirección en R1 y el RC en R15
R13	la *save area*	`STM R14,R12,12(R13)`, la primera instrucción (L01)
R12 R11 R10	**las tres bases del programa**	`USING ZTKMON,R12,R11,R10` (sentencia 10132)
R2 – R9	trabajo, y bases de las áreas de z/OS	`USING SSOB,R2` · `USING CVT,R1` · `USING ENTRADA,R9`

Fuente: Listado ASM de JOB34775: sentencias 10126, 10132, 10138-10148 y 12181

Lámina L15-10_registros

2. Cómo arma la CPU una dirección: base + desplazamiento

Una instrucción que toca memoria no guarda la dirección: guarda **qué registro usar de base** (4 bits) y **cuántos bytes sumarle** (12 bits).

`CLI MODO,C'O'` (sentencia 10202) ensambló como `95D6 BE08`:

Parte	Valor	Qué es
`95`	—	la instrucción CLI («compará un byte»)
`D6`	la O en EBCDIC	el byte con que se compara
`B`	R11	la **base**
`E08`	3.592	el **desplazamiento**

R11 vale el comienzo del programa + X'1000', así que X'1000' + X'E08' = **X'1E08'**: justo `MODO`, el campo de la Clase 2.

Analogía: un edificio de departamentos. La base es el **piso**, y el desplazamiento es la **puerta** dentro de ese piso. Cada piso tiene 4.096 puertas como máximo, porque 12 bits dan 2^{12} = 4.096 números (X'000' a X'FFF).

3. Los tres pisos de ZTKMON, con un ejemplo de cada uno

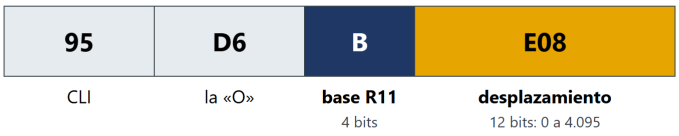
Piso	Base	Cubre	Instrucción real	Bytes	Cuenta
1	R12	X'0000'–X'0FFF'	`B CERRAR` (10189)	`47F0 C27A`	X'0000' + X'27A' = X'027A'
2	R11	X'1000'–X'1FFF'	`CLI MODO,C'O'` (10202)	`95D6 BE08`	X'1000' + X'E08' = X'1E08'
3	R10	X'2000'–X'2FFF'	`MVC SALIDA,ESPACIOS` (10199)	`D284 A466 A4EB`	X'2000' + X'466' = X'2466'

Tres pisos × 4.096 = **12.288 bytes**. Lo que el programa nombra por su etiqueta tiene que caer adentro.

Base + desplazamiento: un ejemplo en cada piso

L15 · Clase 3 · Registros, base y desplazamiento

CLI MODO,C'O' → 95D6 BE08



La CPU suma:
R11 = inicio + X'1000'
+ X'0E08'
= inicio + X'1E08' ← MODO

La instrucción nunca supo la dirección real: sólo «desde R11, 3.592 bytes más allá».

Piso	Base	Cubre	Instrucción real	Bytes	Cuenta
1	R12 (`C`)	X'0000'–X'0FFF'	`B CERRAR`	`47F0 C27A`	X'0000' + X'27A' = X'027A'
2	R11 (`B`)	X'1000'–X'1FFF'	`CLI MODO,C'O'`	`95D6 BE08`	X'1000' + X'E08' = X'1E08'
3	R10 (`A`)	X'2000'–X'2FFF'	`MVC SALIDA,ESPACIOS`	`D284 A466 A4EB`	X'2000' + X'466' = X'2466'

Analogía: el edificio
La base es el **piso** y el desplazamiento es la **puerta**. Cada piso tiene 4.096 puertas, y para llegar a una puerta del piso 3 hace falta alguien parado en el piso 3 (R10).

Fuente: Listado ASM de JOB34775: sentencias 10189 (B CERRAR), 10199 (MVC SALIDA) y 10202 (CLI MODO)

Lámina L15-11_base_desplazamiento

4. El prólogo arma los pisos

Sentencias 10126–10132 de JOB34775:

Sentencia	Fuente	Bytes	Qué hace
10126	`STM R14,R12,12(R13)`	`90EC D00C`	guarda los registros del que llamó; la base es R13 (`D`)
10127	`LR R12,R15`	`18CF`	R15 trae dónde empieza el programa (Clase 1) y R12 lo copia: **piso 1**
10128	`LA R11,4095(,R12)`	`41B0 CFFF`	R11 = R12 + 4.095
10129	`LA R11,1(,R11)`	`41B0 B001`	R11 = R11 + 1, o sea R12 + 4.096: **piso 2**
10130–10131	lo mismo con R10	`41A0 BFFF` · `41A0 A001`	R10 = R11 + 4.096: **piso 3**
10132	`USING ZTKMON,R12,R11,R10`	(no genera bytes)	le **promete al ensamblador** qué vale cada registro

- **Por qué hacen falta dos `LA`:** el desplazamiento de `LA` también es de 12 bits, así que llega como mucho a 4.095

(`FFF`). Para sumar 4.096 hay que sumar 4.095 y después 1.

- **`USING` no se ejecuta.** Es una promesa: «R12 apunta al comienzo de ZTKMON, R11 a +4.096 y R10 a +8.192». Con eso, el

ensamblador calcula base y desplazamiento de cada etiqueta. Si la promesa es falsa, por ejemplo porque alguien pisa R11, el programa lee y escribe en lugares equivocados **sin ningún aviso**.



Lámina L15-12_prologo

5. Cuánto se usó de cada piso: el *Using Map*

El listado trae, al final, cuánto llegó a usar cada base (página 227, columna *Max Disp*):

Base	Piso	Desplazamiento máximo usado	Hasta	*Last Stmt*
R12	1	X'FF2'	X'0FF2'	12492
R11	2	X'FF0'	X'1FF0'	12622
R10	3	**X'873'**	X'2873'	12617

- Los pisos 1 y 2 están **llenos**. Sólo queda lugar en el 3, desde X'2880' hasta X'3000': los **1.920 bytes libres** que

midió la L15.

- **Si algo queda fuera de los tres pisos**, HLASM da `ASMA034E Operand beyond active USING range`, con severidad 8, y esa

instrucción no queda bien armada. Por el `COND=(4,LT,ASM)` de la Clase 1, el binder ni siquiera corre.

- **Nunca pasó todavía:** JOB34775 terminó con RC 0, y ninguna salida guardada en `monitor_z/salidas/` tiene un

`ASMA034E`.

Cuánto se usó de cada piso: el Using Map

L15 · Clase 3 · Registros, base y desplazamiento



piso 1 · lleno | piso 2 · lleno | piso 3 · hasta X'2873'

Arriba de X'3000' no hay piso: una etiqueta ahí da ASMA034E (severidad 8), y por COND=(4,LT,ASM) el binder no corre.

Nunca pasó: JOB34775 dio RC 0 y ninguna salida de monitor_z/salidas tiene un ASMA034E.

Base	Max Disp (Using Map)	Hasta	Última sentencia
R12	`00FF2`	X'0FF2'	12492
R11	`00FF0`	X'1FF0'	12622
R10	`00873`	X'2873'	12617

> **Con módulos** cada CSECT tiene su propio edificio y su propio `USING`: `ZTKFMT` tiene 4.096 bytes para sus ~890 de rutinas.

Fuente: Using Map del listado ASM de JOB34775 (pág. 227) · búsqueda de ASMA034E en monitor_z/salidas

Lámina L15-13_using_map

6. Qué cambia con los módulos

- **Hoy:** un solo edificio de 3 pisos para todo el programa, y 2 de los pisos están llenos.
- **Con módulos:** cada CSECT es su propio edificio, con su propio `USING` y su propia base. `ZTKFMT` tiene 4.096 bytes

para sus ~890 bytes de rutinas.

- **Los datos** dejan de ocupar pisos: se ubican desde un registro que apunta al área común, igual que hoy `USING SSOB,R2`

ubica el bloque de JES (Clase 6).

7. Resumen

Idea	En una línea
Registro	una de 16 manos de 4 bytes; R0, R1, R13, R14 y R15 no están libres
Base + desplazamiento	la dirección = lo que vale el registro + un número de 12 bits (0–4.095)

Una base	cubre 4.096 bytes; ZTKMON tiene 3 bases, o sea 12.288 bytes
`USING`	una promesa al ensamblador; no genera bytes
Using Map	dice cuánto se usó de cada base: R12 y R11 llenas, R10 hasta X'873'
`ASMA034E`	una etiqueta quedó fuera de las bases; severidad 8

8. Preguntas de comprobación

1. `MVC SALIDA,ESPACIOS` ensambló como `D284 A466 A4EB`. ¿Qué base usa para `SALIDA`, con qué desplazamiento, y en qué dirección del programa queda `SALIDA`?
2. ¿Por qué el prólogo arma R11 con dos `LA` (4.095 y después 1) y no con un solo `LA R11,4096(,R12)`?

Mis respuestas

Notas